

GSM ToolPro

Logiciel tout-en-un pour techniciens GSM & maintenance PC

Cahier des charges technique — Architecture, squelette, dépendances, fonctionnalités

Préparé pour : Richard — Cotonou, Bénin

Stack de référence : Electron + Python (Flask) + Supabase + Fedapay/KKiapay

1. Présentation du projet

GSM ToolPro est un logiciel de bureau (Windows, portable techniciens) destiné aux réparateurs GSM et maintenanciers PC. Il centralise en une seule interface : la gestion des interventions clients avec traçabilité, la réinitialisation légitime (reset factory) de téléphones et PC verrouillés, un module ADB/fastboot complet, la reprogrammation/flash de firmware officiels, un diagnostic matériel, un antivirus/nettoyeur intégré, une base de connaissances de solutions par modèle, et un module de facturation.

Principe fondateur : aucune fonction de contournement de sécurité (FRP bypass, déblocage opérateur non autorisé) n'est incluse. Chaque intervention de déblocage passe par une **fiche de vérification du propriétaire** (IMEI, numéro SIM, dernier compte Gmail, pièce d'identité, signature) enregistrée et horodatée, ce qui protège légalement l'atelier et le client.

2. Fonctionnalités & services (vue d'ensemble)

Module	Fonctions principales
Gestion des interventions	Fiche client obligatoire, historique, reçu PDF, statuts, recherche par IMEI/nom
Reset Android	Reset factory ADB/fastboot, détection auto du mode dispo, lecture IMEI, suppression compte Google
Reset PC Windows/Mac	Reset mot de passe local via WinRE, suppression compte Microsoft oublié, sauvegarde avant reset
Module ADB avancé	Gestion fichiers, backup/restauration, terminal shell, install/désinstall APK, logcat live
Reprogrammation / Flash	Flash ROM officielle multi-chipset, vérification checksum, sauvegarde partition anti-brick
Diagnostic matériel	Test batterie, écran tactile, capteurs, caméra, haut-parleurs, réseau/SIM
Antivirus & nettoyage	Scan PC/Android, suppression malware connu, nettoyage fichiers temporaires, quarantaine
Base de connaissances	Procédures par marque/modèle, mise à jour à distance (Supabase), recherche par symptôme
Gestion commerciale	Facturation FedaPay/KKiapay, statistiques, multi-techniciens, licence logicielle
Sécurité & conformité	Journal d'audit non modifiable, chiffrement des pièces d'identité, export légal
Vérification IMEI/blacklist	Contrôle via base GSMA/CheckMEND avant intervention, blocage auto si appareil déclaré volé/perdu
Déblocage réseau opérateur	Appel API vers serveur de codes officiel (constructeur/opérateur), preuve de propriété obligatoire
Réparation logicielle avancée	Bootloop (reflash stock), IMEI corrompu (restauration NVRAM/QCN), baseband unknown (reflash mo
Codes déverrouillage écran	Génération via service officiel constructeur (ex. Samsung Find My Mobile) — pas un bypass
Transfert de données	Migration ancien vers nouveau téléphone via ADB backup/restore ou OTG
Mise à jour firmware OTA manuel	Flash de la dernière version stock officielle quand l'appareil ne se met pas à jour seul

3. Architecture technique globale

Application desktop **Electron** (frontend HTML/JS/CSS) pilotant un backend local **Python/Flask** exposé sur localhost, qui gère les appels système (ADB, fastboot, scan antivirus) et la communication avec **Supabase** (base de données + auth + storage) pour la synchronisation cloud, l'historique client et les mises à jour de la base de connaissances. La facturation utilise l'API FedaPay/KKiapay, comme sur tes autres projets (Allô Livreur, APK Factory Pro).

Schéma des flux

```
[Interface Electron] <--IPC--> [main.js]
    |
    v (HTTP localhost:5000)
[Flask API - server.py]
|-- adb_manager.py          --> binaire adb.exe (USB)
|-- fastboot_manager.py     --> binaire fastboot.exe (USB)
|-- device_detect.py        --> pyusb / getprop
|-- reset_module.py         --> orchestration reset
|-- boot_module.py          --> reboot recovery/bootloader
|-- antivirus_module.py     --> scan + suppression menaces
|-- firmware_db.py          --> lecture/écriture Supabase
|-- billing.py              --> FedaPay / KKiapay
|-- license_check.py        --> validation licence Supabase
    |
    v
[Supabase Cloud]
- table interventions
- table clients
- table solutions_db
- table licences
- storage: pieces_identite, signatures
```

4. Squelette complet du projet (arborescence)

```
GSM-ToolPro/
■■■ package.json
■■■ main.js                # Electron main process
■■■ preload.js             # Bridge sécurisé IPC
■■■ icon.ico
■■■
■■■ backend/
■   ■■■ server.py          # Point d'entrée Flask
■   ■■■ requirements.txt
■   ■■■ config.py          # Clés Supabase / FedaPay (via .env)
■   ■■■ .env.example
■   ■
■   ■■■ modules/
■   ■   ■■■ __init__.py
■   ■   ■■■ adb_manager.py
■   ■   ■■■ fastboot_manager.py
■   ■   ■■■ device_detect.py
■   ■   ■■■ reset_module.py
■   ■   ■■■ boot_module.py
■   ■   ■■■ flash_module.py
■   ■   ■■■ diagnostic_module.py
■   ■   ■■■ antivirus_module.py
■   ■   ■■■ imei_check_module.py # Vérif blacklist GSMA/CheckMEND
■   ■   ■■■ carrier_unlock_module.py # API serveur codes opérateur
■   ■   ■■■ advanced_repair_module.py # Bootloop/IMEI/baseband
■   ■   ■■■ screen_unlock_module.py # API codes constructeur officiels
```

```

■ ■ ■■■ data_transfer_module.py # Migration ADB backup/restore
■ ■ ■■■ ota_update_module.py    # Flash firmware stock manuel
■ ■ ■■■ firmware_db.py
■ ■ ■■■ intervention_manager.py
■ ■ ■■■ billing.py
■ ■ ■■■ license_check.py
■ ■ ■■■ pdf_generator.py        # Reçus/rapports d'intervention
■ ■
■ ■■■ database/
■ ■■■ schema.sql                # Schéma Supabase complet
■
■■■ frontend/
■ ■■■ index.html                # Dashboard principal
■ ■■■ nouvelle-intervention.html
■ ■■■ fiche-verification.html
■ ■■■ adb-panel.html
■ ■■■ fastboot-panel.html
■ ■■■ flash-panel.html
■ ■■■ diagnostic.html
■ ■■■ antivirus.html
■ ■■■ solutions-db.html
■ ■■■ facturation.html
■ ■■■ historique-client.html
■ ■■■ licence.html
■ ■■■ css/
■ ■ ■■■ style.css
■ ■■■ js/
■ ■■■ api.js                    # Wrapper fetch vers Flask
■ ■■■ device-monitor.js        # Détection USB en temps réel
■ ■■■ ui-helpers.js
■
■■■ bin/
■ ■■■ adb.exe
■ ■■■ fastboot.exe
■ ■■■ firmware/                # ROMs officielles téléchargées
■
■■■ build/
■ ■■■ electron-builder.yml      # Config packaging .exe

```

5. Dépendances complètes

5.1 Frontend / Electron (package.json)

```
{
  "name": "gsm-toolpro",
  "version": "1.0.0",
  "main": "main.js",
  "scripts": {
    "start": "electron .",
    "build": "electron-builder"
  },
  "dependencies": {
    "electron-store": "^8.2.0",
    "node-fetch": "^3.3.2",
    "usb-detection": "^4.14.2",
    "pdf-lib": "^1.17.1"
  },
  "devDependencies": {
    "electron": "^30.0.0",
    "electron-builder": "^24.13.3"
  }
}
```

5.2 Backend Python (requirements.txt)

```
flask==3.0.3
flask-cors==4.0.1
supabase==2.5.0
python-dotenv==1.0.1
pyusb==1.2.1
reportlab==4.2.0      # génération reçus PDF
pillow==10.3.0        # traitement photos pièce d'identité
requests==2.32.3      # appels FedaPay/KKiapay
psutil==5.9.8         # infos système diagnostic PC
pyclamd==0.4.0        # intégration antivirus ClamAV
cryptography==42.0.5  # chiffrement pièces d'identité
```

5.3 Outils système externes à embarquer

- **ADB & Fastboot** (Android SDK Platform Tools) — officiels Google, gratuits, redistribuables
- **ClamAV** (moteur antivirus open-source) pour le scan PC — évite de recoder un antivirus
- **7-Zip CLI** pour extraction de firmware compressés
- **OpenSSL** pour vérification de signatures/checksums de firmware

5.4 Base de données Supabase — schéma principal

```
-- Table interventions (fiche de vérification)
create table interventions (
  id uuid primary key default gen_random_uuid(),
  technicien_id uuid references profiles(id),
  client_nom text not null,
  client_telephone text not null,
  sim_numero text,
  imei text,
  dernier_gmail text,
  piece_identite_url text,
  type_appareil text check (type_appareil in ('android','pc','ios')),
  marque_modele text,
  motif text default 'oubli code',
```

```

    statut text default 'en_cours',
    signature_url text,
    montant_facture numeric,
    created_at timestamptz default now()
);

-- Table base de solutions
create table solutions_db (
    id uuid primary key default gen_random_uuid(),
    marque text, modele text, chipset text,
    symptome text, procedure text, outils_requis text,
    created_at timestamptz default now()
);

-- Table licences logiciel
create table licences (
    id uuid primary key default gen_random_uuid(),
    technicien_id uuid references profiles(id),
    statut text default 'actif',
    date_expiration date,
    created_at timestamptz default now()
);

-- Journal d'audit non modifiable
create table audit_log (
    id uuid primary key default gen_random_uuid(),
    technicien_id uuid, action text, details jsonb,
    created_at timestamptz default now()
);

```

6. Module antivirus & nettoyage

Plutôt que de recréer un moteur de détection de virus (tâche énorme et peu fiable si fait seul), **GSM ToolPro** intègre **ClamAV** (moteur open-source reconnu, base de signatures mise à jour quotidiennement) piloté via pyclamd. Cette approche est standard dans l'industrie.

- Scan complet ou rapide du PC/disque du client
- Scan d'un téléphone Android connecté (analyse des APK installés via ADB, détection d'apps suspectes)
- Mise en quarantaine automatique des fichiers infectés (déplacement, pas suppression directe)
- Nettoyage des fichiers temporaires, cache navigateur, résidus d'installation
- Rapport de scan exportable en PDF pour le client
- Mise à jour automatique des signatures virales au démarrage du logiciel

7. Fiche de vérification client (avant tout reset)

Étape obligatoire et non contournable dans le workflow logiciel avant d'autoriser un reset factory :

- Nom complet et numéro de téléphone du client
- Numéro de la carte SIM insérée dans l'appareil (si disponible)
- IMEI de l'appareil (lecture automatique via *#06# ou ADB si accessible)
- Dernier compte Gmail utilisé sur l'appareil (déclaratif du client)
- Photo de la pièce d'identité (optionnelle mais recommandée, stockée chiffrée)
- Signature électronique du client (décharge de responsabilité)

Une fois validée, la fiche est enregistrée avec horodatage dans Supabase et un **reçu PDF** est généré automatiquement (via pdf_generator.py + reportlab), remis au client comme preuve légale de l'intervention.

8. Services numériques complémentaires

8.1 Vérification IMEI / blacklist (garde-fou prioritaire)

Ce contrôle s'exécute **avant même la fiche de vérification**, dès qu'un appareil est détecté. Le logiciel interroge une base type GSMA Device Registry ou CheckMEND via API pour savoir si l'IMEI est déclaré volé/perdu. Si c'est le cas, le logiciel affiche une alerte au technicien et peut bloquer la poursuite de l'intervention selon la politique de l'atelier.

8.2 Déblocage réseau opérateur

Le déblocage ne se fait **pas** par exploit local mais via l'envoi de l'IMEI à un **serveur de codes officiels** (service payant à l'acte, ex. type DirectUnlocks/UnlockBase) qui retourne un code de déverrouillage valide auprès du constructeur/opérateur. Le logiciel se contente d'orchestrer cet appel API et d'exiger une preuve de propriété (facture ou déclaration signée) avant l'envoi.

8.3 Réparation logicielle avancée

- **Bootloop** : réinstallation de la ROM stock officielle + wipe cache/dalvik
- **IMEI corrompu/null** : restauration depuis une sauvegarde NVRAM/QCN préalablement effectuée par le technicien
- **Baseband unknown** : reflash du modem/baseband depuis le firmware constructeur officiel

8.4 Codes de déverrouillage d'écran via service constructeur

Utilise les services officiels des constructeurs (ex. Samsung Find My Mobile, compte Xiaomi Mi Cloud avec identifiants du propriétaire) — le technicien se connecte avec les identifiants du client pour déclencher un déverrouillage légitime à distance, jamais un contournement technique.

8.5 Transfert de données entre téléphones

Migration ancien vers nouveau téléphone via ADB backup/restore, ou transfert direct par câble OTG, incluant contacts, photos, messages et applications compatibles.

8.6 Mise à jour firmware/OS manuelle (OTA)

Pour les appareils bloqués sur une ancienne version à cause d'un problème de mise à jour automatique, flash manuel de la dernière version stock officielle correspondant au modèle exact.

9. Roadmap de développement suggérée

Phase	Contenu	Durée estimée
1	Squelette Electron+Flask, fiche de vérification, base Supabase	1-2 semaines
2	Vérification IMEI/blacklist (garde-fou avant intervention)	3-4 jours
3	Module ADB complet (backup, shell, install, logs)	1 semaine
4	Module fastboot + reset factory Android	1 semaine
5	Module reset PC Windows (WinRE)	1 semaine
6	Diagnostic matériel + base de solutions	1-2 semaines
7	Antivirus (intégration ClamAV) + nettoyage	1 semaine
8	Facturation FedaPay/KKiapay + licence logicielle	1 semaine
9	Flash/reprogrammation multi-chipset (module avancé)	2 semaines
10	Débloccage opérateur (intégration serveur de codes tiers)	1 semaine
11	Réparation avancée (bootloop, IMEI, baseband) + transfert data + OTA	1-2 semaines
12	Tests terrain, packaging .exe, distribution	1 semaine

Document généré comme base de travail. Prochaine étape : génération du code source du squelette (main.js, server.py, schema.sql, premières pages HTML) pour démarrer le développement.